SolarShare — COMPLETE MASTER PLAN (Final, Unified, v6)

Yeh EK hi file hai jo sab kuch cover karti hai — GitHub, architecture, aur har member ka exact step-by-step kaam. Koi aur file dekhne ki zaroorat nahi.

📜 THE 5 GOLDEN RULES (poori team ko yeh pehle padhna hai)

1. **Folder ownership:** M4 sirf `design/` mein likhega. M5 sirf `app/` mein likhega. M6 sirf `backend/` mein likhega. M3 sirf `data/` mein likhega. Kabhi bhi do log ek hi folder/file ko save nahi karenge.
2. **Hour 3 ke baad, M4 sirf CSS properties badal sakta hai** (`design/styles.css` mein) — HTML tags/nesting/structure bilkul nahi, chahe Stitch re-generate karke bhi nahi.
3. **App kabhi double-click se nahi kholna** — hamesha VS Code Live Server se hi (`http://localhost` URL), warna data-fetch fail hoga.
4. **Koi external CDN link nahi** (fonts/libraries) — sirf browser-native APIs, taaki venue wifi slow ho toh bhi app chale.
5. **Har 30-45 min mein GitHub commit + push karo** — chhote frequent commits, aakhri mein ek bada dump nahi.

🔒 THE SHARED CONTRACT (yeh sabki "common language" hai)

Data (M3 banayega, sab isi ko use karenge)

```
{
  "users": [
    { "id": "U1", "name": "Priya",  "unit": "A-302", "role": "prosumer", "surplus_kwh": 5.0,
"asking_price": 6.0 },
    { "id": "U2", "name": "Rahul",  "unit": "D-201", "role": "buyer",    "demand_kwh": 3.0,
"max_price": 6.5 },
    { "id": "U3", "name": "Sanjay", "unit": "B-105", "role": "prosumer", "surplus_kwh": 4.2,
"asking_price": 5.8 },
    { "id": "U4", "name": "Meera",  "unit": "C-401", "role": "buyer",    "demand_kwh": 2.5,
"max_price": 6.0 },
    { "id": "U5", "name": "Vikram", "unit": "E-501", "role": "buyer",    "demand_kwh": 3.5,
"max_price": 6.2 }
  ]
}
```

JSON key spellings exactly yehi honi chahiye — `surplus_kwh` , `asking_price` , `demand_kwh` , `max_price` — M5 aur M6 dono, koi variation nahi (camelCase mat karna).

The Exact Matching Algorithm (M5 aur M6 dono apne AI prompt mein yeh EXACT block paste karenge)

```
1. Sort all open listings by asking_price ascending; if tied, sort by listing seller's id
ascending.
```

```
2. Sort all buyers by max_price descending; if tied, sort by buyer's id ascending.
3. For each buyer in this sorted order:
   a. Find the cheapest available listing where listing.asking_price <= buyer.max_price.
   b. If found: kwh_traded = min(listing.remaining_kwh, buyer.remaining_demand).
   c. Reduce listing.remaining_kwh and buyer.remaining_demand by kwh_traded.
   d. If listing.remaining_kwh > 0, it stays available for the next buyer.
   e. If no listing satisfies buyer.max_price, that buyer stays unmatched.
4. Continue until all buyers are processed.
```

Yeh procedure Hour 0 mein poori team ke saamne zor se padho, confirm karo sab samjhe — isse hi guaranteed same output milega JS aur Python dono mein.

Hash Logic (dono JS aur Python mein same concept)

```
combined_string = JSON.stringify(trade_data_without_hash_field) + previous_hash
hash = SHA256(combined_string)
```

HTML Element IDs (M4 exactly yeh use karega, M5 exactly yehi dhoondega)

Screen	Element	ID
Dashboard	kWh input	`surplus-kwh-input`
Dashboard	Price input	`asking-price-input`
Dashboard	List button	`list-surplus-btn`
Dashboard	Screen container	`screen-dashboard`
Marketplace	Listings container	`marketplace-listings`
Marketplace	Card template	`listing-card-template` (with `data-listing-id`)
Marketplace	Buy button	class `buy-btn` (with `data-listing-id`)
Marketplace	Screen container	`screen-marketplace`
Ledger	Table body	`ledger-table-body`
Ledger	Row template	`ledger-row-template`
Ledger	Screen container	`screen-ledger`
Nav	3 nav buttons	`nav-dashboard`, `nav-marketplace`, `nav-ledger`

Repo/Folder Structure

```
solarshare-prototype/
├── data/dataset.json                          ← M3
├── design/dashboard.html, marketplace.html,
│         ledger.html, styles.css              ← M4
├── app/index.html, script.js                  ← M5 (fetches directly from ../backend/)
└── backend/matching_engine.ipynb,
            backend_output.json                ← M6
```

STEP 0 — SABSE PEHLE (poori team, ek saath, hackathon shuru hone se pehle)

1. GitHub par account, repo `solarshare-prototype` banao, **Private**.
2. Upar wali 4-folder structure banao.
3. Baaki 5 members ko collaborator add karo. Sabhi **GitHub Desktop** install karke test-push kar lein.
4. Poori team ke saamne **Matching Algorithm block** aur **ID table** zor se padho — sab confirm karein samajh gaye.
5. Golden Rules (upar) sabko explicitly bata do, especially Rule #2 (Hour 3 cutoff) aur Rule #3 (Live Server only).

👤 M3 — Data & QA Lead

1. **Hour 0-1:** Google Sheets app (phone) mein upar wala dataset enter karo.
2. `data/dataset.json` banao/commit karo. **Commit se pehle** ek online JSON validator (jsonlint.com) se check karo valid JSON hai.
3. Commit: `"Add household dataset"`, Push.
4. **Hour 1:** M4 ke Stitch prompts mein numbers verify karo — sheet se exactly match hone chahiye.
5. **Hour 5-6:** Jaise hi M5 ka basic navigation chalu ho, phone se test shuru karo. Bugs turant group mein batao.
6. **Hour 8:** Poora working flow ka screen-recording video banao (backup demo).
7. **Hour 14:** Agar Hour 13 ke dry run mein koi fix hua ho, video **dobara record karo** — purana outdated video kabhi use mat karna.
8. Ek green/red checklist banao M1/M2 ke liye — exact items yeh honi chahiye (test karke ✅/❌ maako):

```
[ ] Dashboard load hote hi Priya ka 5.0 kWh surplus sahi dikh raha hai
[ ] "List Surplus" button naya listing Marketplace mein add karta hai
[ ] Marketplace mein saari open listings sahi seller/price ke saath dikh rahi hain
[ ] "Buy" button click karne se listing status update hoti hai
[ ] Trade complete hone ke baad ek real hash dikhta hai (blank ya "undefined" nahi)
[ ] Ledger mein naya trade sahi kWh/price/hash ke saath dikh raha hai
[ ] Doosre trade ka "previous_hash" pehle trade ke "hash" se exactly match karta hai
[ ] Teeno screens ke beech navigation dono directions mein kaam karta hai
[ ] Ek hi listing pe do baar fast "Buy" click karne se app crash/break nahi hota AUR sirf ek hi
trade banta hai (button turant disable ho jana chahiye, dusra click kaam nahi karna chahiye)
[ ] Backend se data load ho raha hai (fallback pe nahi gira, jab tak backend file missing na
ho)
```

9. "Reset Demo" button (M5 banayega) ke baare mein M1/M2 ko bata do — agar kuch atke, ghabrao mat, woh button dabao.

M3's exact dataset copy-paste block (team ko WhatsApp/chat mein bhejne ke liye)

```
DATASET (use exactly these numbers everywhere):
Priya (A-302) — Prosumer — 5.0 kWh surplus — asking ₹6.0/unit
Rahul (D-201) — Buyer — needs 3.0 kWh — willing to pay up to ₹6.5/unit
Sanjay (B-105) — Prosumer — 4.2 kWh surplus — asking ₹5.8/unit
Meera (C-401) — Buyer — needs 2.5 kWh — willing to pay up to ₹6.0/unit
Vikram (E-501) — Buyer — needs 3.5 kWh — willing to pay up to ₹6.2/unit
```

```
DISCOM reference for comparison: Feed-in tariff ₹4.10/unit | Retail tariff ₹6.00–8.00/unit
```

🎨 M4 — Google Stitch (UI Code)

1. **Hour 1:** `stitch.withgoogle.com` par yeh prompt do:

```
Create a mobile-first web app screen called "Dashboard" for a solar energy trading app called SolarShare.
Design: Dark navy background (#0B1626), amber accent (#FFB347) for seller/earning elements, teal (#3ED9A8) for confirmed elements, light text (#EAF0FB). Card-based, single column, calm — not a flashy fintech look.
Content:
- Header: "Good morning, Priya"
- One card: "Today's Surplus: 5.0 kWh available", suggested price ₹6.0/unit
- Input field id="surplus-kwh-input" pre-filled 5.0
- Input field id="asking-price-input" pre-filled 6.0
- Amber button id="list-surplus-btn" labeled "List This Surplus"
- Bottom nav: id="nav-dashboard", id="nav-marketplace", id="nav-ledger"
- Wrap everything in container id="screen-dashboard"
Export as clean HTML and CSS.
```

2. Export, **self-check IDs** using the ID table (Ctrl+F in the file) — agar koi missing/different, khud fix karo before sending.
3. Save `design/dashboard.html`. Commit: `"Add Stitch Dashboard screen"`, Push.
4. **Hour 2:** Same for Marketplace:

```
Create screen "Marketplace" for SolarShare (same colors: navy #0B1626, amber #FFB347, teal #3ED9A8).
- Header: "Available Nearby"
- Container id="marketplace-listings"
- Card template id="listing-card-template": seller unit, price/kWh, kWh available, teal button class="buy-btn" with data-listing-id
- Empty state: "No listings nearby — you're on standard grid"
- Same bottom nav
- Wrap in container id="screen-marketplace"
Export as clean HTML and CSS.
```

Self-check IDs, save `design/marketplace.html`, commit, push.
5. **Hour 3:** Same for Ledger:

```
Create screen "Ledger" for SolarShare (same colors).
- Header: "Trade History"
- Container id="ledger-table-body"
- Row template id="ledger-row-template": date, counterparty, kWh, price, hash (monospace, teal, small)
- Empty state: "No trades yet."
- Same bottom nav
- Wrap in container id="screen-ledger"
Export as clean HTML and CSS.
```

Self-check IDs, save `design/ledger.html`, commit, push.
6. ⚠️ **Hour 3 ke baad:** ab sirf `design/styles.css` mein CSS property changes — HTML structure bilkul nahi. Agar structural change chahiye, M5 se seedha baat karo.
7. **Hour 9-10 (agar time bache):** M5 ke flagged visual issues ke liye CSS tweak karo, commit: `"Polish [screen] spacing/colors"`, push.

⚙️ M5 — Antigravity/VS Code (Logic Layer)

1. **Hour 2 (M4 ka Dashboard aate hi):** Code copy karo `app/index.html` mein. M4 ki original file edit mat karo.
2. ID table se turant verify karo — mismatch mile toh khud manually fix karo.
3. Antigravity ko yeh poora prompt do (sab kuch ek saath):

```
I have existing HTML/CSS for a 3-screen web app called SolarShare with these exact element IDs:
surplus-kwh-input, asking-price-input, list-surplus-btn, screen-dashboard, marketplace-
listings,
listing-card-template, buy-btn (class), screen-marketplace, ledger-table-body, ledger-row-
template,
screen-ledger, nav-dashboard, nav-marketplace, nav-ledger.

Starting data:
const users = [
  { id: "U1", name: "Priya", unit: "A-302", role: "prosumer", surplus_kwh: 5.0, asking_price:
6.0 },
  { id: "U2", name: "Rahul", unit: "D-201", role: "buyer", demand_kwh: 3.0, max_price: 6.5 },
  { id: "U3", name: "Sanjay", unit: "B-105", role: "prosumer", surplus_kwh: 4.2, asking_price:
5.8 },
  { id: "U4", name: "Meera", unit: "C-401", role: "buyer", demand_kwh: 2.5, max_price: 6.0 },
  { id: "U5", name: "Vikram", unit: "E-501", role: "buyer", demand_kwh: 3.5, max_price: 6.2 }
];
let listings = []; // will be loaded from backend
let fallbackListings = [
  { listing_id: "L1", seller_id: "U1", kwh_available: 5.0, price_per_kwh: 6.0, status: "open"
},
  { listing_id: "L2", seller_id: "U3", kwh_available: 4.2, price_per_kwh: 5.8, status: "open" }
];
let trades = [];
let lastHash = "0000000000000000";

Required functions:

1. loadInitialData() — async function. Try to fetch('../backend/backend_output.json?v=' +
Date.now()), parse it, and set listings = data.listings. If the fetch fails (use try/catch),
log a warning and set listings = fallbackListings instead. Then call renderMarketplace(). Call
this on window.onload.

2. renderMarketplace() — clone #listing-card-template per open listing, fill seller/price/kWh,
attach click on .buy-btn calling buyListing(listing_id).

3. buyListing(listing_id) — ⚠️ IMPORTANT — the .buy-btn that was clicked MUST be disabled
(button.disabled = true) as the VERY FIRST line of this function, before any async work starts.
This prevents a double-click race condition where two rapid clicks could both read the same
lastHash before either finishes updating it, corrupting the hash chain. Only re-enable relevant
buttons after renderMarketplace() and renderLedger() are called at the end of this function.
Then: find listing, build trade object {trade_id, listing_id, seller_id, buyer_id:"U2",
```

```
kwh_traded, price_per_kwh, timestamp: new Date().toISOString(), previous_hash: lastHash}. Call
`await generateHash(tradeData)` (must use await — this is an async function), attach the result
as trade.hash, push to trades[], mark listing status "matched", update lastHash, re-render
marketplace and ledger.

4. generateHash(tradeData) — MUST be an async function. Use `await crypto.subtle.digest('SHA-
256', ...)` on JSON.stringify(tradeData without hash field) + previous_hash. Convert the
ArrayBuffer result to a hex string. Return it. Do NOT call this function without await
anywhere.

5. renderLedger() — clone #ledger-row-template per trade, show date/counterparty/kWh/price/hash
(first 12 characters + "...").

6. listSurplus() — on #list-surplus-btn click, read #surplus-kwh-input and #asking-price-input
values, push new listing object, call renderMarketplace().

7. Navigation — nav-* buttons use simple style.display = 'block' / 'none' to show one screen-*
container and hide the other two.

8. resetDemo() — resets listings, trades, and lastHash back to their initial state, then re-
renders all screens. Attach this to a small, discreet button (not prominently placed) for demo
safety.

Plain vanilla JavaScript only, no frameworks, no external CDN links or libraries — everything
must work fully offline once the page is loaded via a local server.
```

3.5. ⚙️ **Matching function — DO NOT let AI regenerate this part fresh.** LLM-generated code isn't guaranteed byte-identical each time, and this is the one function where JS and Python MUST behave identically. Paste this **exact, pre-verified** matching function directly into your code instead of asking Antigravity to write it:

```
function matchListingsToBuyer(buyer, listings) {
    // Sort available listings by price ascending, tie-break by seller id ascending
    const available = listings
        .filter(l => l.status === "open" && l.price_per_kwh <= buyer.max_price)
        .sort((a, b) => {
            if (a.price_per_kwh !== b.price_per_kwh) return a.price_per_kwh - b.price_per_kwh;
            return a.seller_id.localeCompare(b.seller_id);
        });
    return available.length > 0 ? available[0] : null;
}
```

4. VS Code Live Server se test karo — Dashboard khul rahi hai?
5. Commit: `"Add core JS logic — hashing, matching, reset function"`, push.
6. **Hour 3-4 (M4 ka Marketplace/Ledger aate hi):** Integrate karo, test karo. Commit: `"Integrate Marketplace and Ledger screens"`, push.
7. **Hour 6-7 (M6 ka backend_output.json ready hone ke baad):** `loadInitialData()` function test karo — listings ab backend se aani chahiye, hardcoded se nahi. Fallback bhi test karo (jaan-boojh kar backend file temporarily rename karke dekho, app crash nahi hona chahiye, fallback data dikhna chahiye).
8. Commit: `"Connect frontend to backend_output.json with fallback"`, push.
9. **Hour 5-6:** Navigation bugs fix karo. Commit: `"Fix navigation logic"`, push.
10. **Hour 7+:** M3 ke bataye bugs fix karo ek-ek, har fix ke baad chhota commit.
11. **Offline test:** Wifi/mobile-data OFF karke bhi app test karo — sab kaam karna chahiye (koi CDN dependency nahi).

12. **Hour 13 (dry run ke baad):** Final polish, commit: `"Final version ready for presentation"`, push.

🐍 M6 — Colab/Python (Backend Engine)

1. **Hour 1 (M4/M5 se independent, turant shuru):** Google Colab kholo, naya notebook.
2. Yeh poora prompt do:

```
Write a Python script for Google Colab using networkx and hashlib.

1. Define this data:
users = [
    {"id": "U1", "name": "Priya", "role": "prosumer", "surplus_kwh": 5.0, "asking_price": 6.0},
    {"id": "U2", "name": "Rahul", "role": "buyer", "demand_kwh": 3.0, "max_price": 6.5},
    {"id": "U3", "name": "Sanjay", "role": "prosumer", "surplus_kwh": 4.2, "asking_price":
5.8},
    {"id": "U4", "name": "Meera", "role": "buyer", "demand_kwh": 2.5, "max_price": 6.0},
    {"id": "U5", "name": "Vikram", "role": "buyer", "demand_kwh": 3.5, "max_price": 6.2},
]

2. Implement this EXACT matching procedure (do not use a different algorithm):
   a. Sort all prosumers (sellers) by asking_price ascending; if tied, sort by id ascending.
   b. Sort all buyers by max_price descending; if tied, sort by id ascending.
   c. For each buyer in this order, find the cheapest available seller where
seller.asking_price <= buyer.max_price.
      If found, trade kwh = min(seller.remaining_kwh, buyer.remaining_demand). Reduce both
remaining amounts.
      If seller still has remaining_kwh > 0, keep them available for the next buyer.
   d. Continue until all buyers are processed.

3. Also build a networkx graph (nodes = users, edges between compatible prosumer-buyer pairs
weighted by asking_price) purely for visualization — draw it with networkx.draw().

4. For each match from step 2, build a trade record:
   trade = {"trade_id": f"T{i}", "seller_id":..., "buyer_id":..., "kwh_traded":...,
"price_per_kwh":...,
            "timestamp": current ISO timestamp, "previous_hash": last_hash}

5. Hash function:
   import hashlib, json
   def generate_hash(trade_data, previous_hash):
       combined = json.dumps(trade_data, sort_keys=True) + previous_hash
       return hashlib.sha256(combined.encode()).hexdigest()
   Chain each trade's hash to the previous one, starting previous_hash = "0000000000000000".

6. Print a clean summary table of all matches with kWh, price, and hash (first 12 chars).

7. Finally, export the result as JSON in this exact structure and save it as
backend_output.json:
{
  "listings": [ {"listing_id": "L1", "seller_id": "U1", "kwh_available": 5.0, "price_per_kwh":
6.0, "status": "open"}, ... ],
  "trades": [ ...matched trade records with hashes... ]
}
Use json.dump() to write this file.
```

```
Well-commented code, may be shown live to judges.
```

⚙️ **Step 2 (matching logic) AI se fresh generate mat karwao** — yeh exact, pre-verified function directly paste karo Colab mein (JS version se guaranteed identical hai, kyunki AI-generated code har baar thoda alag aa sakta hai):

```
def match_listing_to_buyer(buyer, listings):
    # Sort available listings by price ascending, tie-break by seller_id ascending
    available = sorted(
        [l for l in listings if l["status"] == "open" and l["price_per_kwh"] <=
buyer["max_price"]],
        key=lambda l: (l["price_per_kwh"], l["seller_id"])
    )
    return available[0] if available else None
```

Baaki poora script (data setup, hash function, graph visualization, JSON export) AI se generate karwa sakte ho — sirf yeh ek matching function manually paste karo.
3. Run karo, output check karo (5 users, sensible matches).
4. `backend_output.json` download karo, save `backend/backend_output.json` mein.
5. Notebook save karo `backend/matching_engine.ipynb`.
6. Commit: `"Add matching engine + exported backend_output.json"`, push.
7. **Hour 6:** M5 ke saath output compare karo — same matched pairs aane chahiye (algorithm exact same hai, toh match hona chahiye). Agar mismatch mile, dono taraf procedure recheck karo — line-by-line compare karo algorithm ke saath.
8. **Hour 10:** Notebook "Restart and run all" karke confirm 0 errors.
9. **Presentation se 15-20 min pehle:** Colab notebook ek baar phir "Run all" karo — session fresh rakhne ke liye.
10. Demo laptop pe Colab tab khula rakho (Q&A backup ke liye, live demo ka part nahi).

🗣️ M1 (Pitcher) & 🛡️ M2 (Domain Defender)

Inka detailed, hour-by-hour, research-backed plan ab alag file mein hai:
`SolarShare_M1_M2_Pitch_Defense_Plan.md`

Dono files saath istemal karo — yeh (COMPLETE_MASTER) technical build ke liye hai (M3-M6), doosri file pitch aur Q&A defense ke liye hai (M1-M2). Ek-dusre ko replace nahi karti, dono zaroori hain.

Quick reference (demo-cue timing sync ke liye, jo dono files ko jodta hai): M1 ka demo-narration window Hour 2:30–4:00 pe hai apne script mein — is waqt M6 (jo doosri file mein detail nahi hai, upar M6 section mein hai) verbal + physical cues sunke live app pe click karega. Poora cue-script doosri file mein hai.

🛡️ DAY-OF CONTINGENCY CHECKLIST (subah sabse pehle yeh check karo)

```
[ ] App SIRF Live Server se open ho raha hai, kabhi double-click se nahi
[ ] Fallback data code test kiya (backend file temporarily hata ke dekha — app crash nahi hua)
[ ] Matching algorithm dono JS aur Python mein exact same procedure follow karta hai (tie-
breaker sahit)
[ ] Cache-busting query param code mein hai
[ ] Backup video Hour 14 ka latest version hai (Hour 8 wala nahi agar fixes hue)
[ ] Reset Demo button test kiya, kaam kar raha hai
[ ] M6's laptop charged + charger saath, second backup laptop repo-cloned ready
```

```
[ ] Colab "Run all" last 20 min mein kiya gaya
[ ] Sab tabs pehle se khule: app, Colab, backup video, GitHub commits page, DevTools Network
tab
[ ] M1-M6 ne verbal+physical dual-cue script 3+ baar practice kiya hai
[ ] Ek cue-skip recovery drill practice kiya gaya hai
[ ] Mobile hotspot backup ready
[ ] Har member ko apna exact role pata hai
```

🎯 ONE-LINE SUMMARY (poori pipeline)

M3 ka verified dataset → M4 ka ID-checked, Hour-3-locked Stitch code (`design/`) → M5 usi code pe live logic banata hai (`app/`) jisme exact matching algorithm + real backend-fetch + fallback + async-safe hashing + button-locking + reset button hai → M6 parallel mein wahi pre-verified matching function Python mein implement karke `backend_output.json` export karta hai (`backend/`) → M5 ka app genuinely woh file fetch karta hai → poori team har 30-45 min GitHub commit karti hai apne-apne folder mein → M1 dual-cue script se demo pitch karta hai → M2 sirf pooche jaane par technical proof (Network tab) dikha ke real architecture defend karta hai.